İZMİR EKONOMİ ÜNİVERSİTESİ

SE 311-Livestock Farm Project Report

Mehmet Taşoğlu - 20210601059

Berkay Işık - 20190601204

Derin Buldanlı - 20210601012

Ayşe Özer - 20200601040

Table of Contents

1. Design Patterns Used

    1.1 Abstract Factory Pattern

    1.2 Observer Pattern

    1.3 Visitor Pattern

    1.4 Singleton Pattern

    1.5 Adapter Pattern

2. UML Diagram

3. Explanation of Classes in Livestock Farm Project

    3.1 Abstract Factory Pattern

    3.2 Observer Pattern

    3.3 Visitor Pattern

    3.4 Singleton Pattern

    3.5 Adapter Pattern

1. Design Patterns Used:

1.1 Abstract Factory Pattern:

- **Why we chose this pattern:**

The Abstract Factory model works well in this cattle farming system because it uses different types of feed for dairy and beef cattle. Dairy cattle need corn and soybeans, while beef cattle need wheat and canola. This model creates custom factories that make sure each cattle gets the right combination of feed. It keeps the code organized and makes it easy to add new cattle breeds that have different traits and eat different types of food.

- **How we mapped participants:**
    1. **Abstract Products**: Carbohydrate and Protein classes.
    2. **Concrete Products**: Corn, Wheat, Soybean, and Canola classes.
    3. **Abstract Factory**: CattleFeedFactory abstract class.
    4. **Concrete Factories**: DairyCattleFeedFactory and BeefCattleFeedFactory.
    5. **Client**: Cattle abstract class using the factories.

1.2 Observer Pattern:

- **Why we chose this pattern:**

We need real- time notifications to notify the farmer when a cattle crosses the farm boundaries. This pattern enables automatic notifications when a state of an object changes. It creates a one-to-many dependency between objects, meaning that when the state of one Cattle changes, all registered Farmer observers are immediately notified. The push method is used in this pattern.

- **How we mapped participants:**
    1. **Subject**: Subject abstract class extended by Cattle
    2. **Observer**: Observer interface
    3. **Concrete Observer**: Farmer class
    4. **Concrete Subject**: Cattle class and its subclasses

1.3 Visitor Pattern:

- **Why we chose this pattern:**

The visitor model is suitable for this farm system because it allows different people to do their work with the cattle without making changes to the cattle code. When the vet comes to vaccinate in the fall or the inspector checks ear tags in the spring, each can handle both dairy and beef cattle. This model keeps the cattle code simple while allowing new types of visitors to be added as needed. Each cattle only needs the admission method that allows visitors to do their specific work with that cattle. Double dispatch is used in this pattern.

- **How we mapped participants:**
    1. **Visitor**: Visitor interface
    2. **Concrete Visitors**: VeterinaryPhysician and MinistryInspector classes
    3. **Element**: Cattle abstract class with accept() method
    4. **Concrete Elements**: DairyCattle and BeefCattle classes

1.4. Singleton Pattern:

- **Why we chose this pattern:**

The singleton pattern is suitable for a farm's location database because it only requires one database to track all cattle locations. When cattle move, their new locations are updated in

this single database. Everyone who needs location information uses the same database via the `getInstance()` method. This keeps all location data in one place. It also checks if cattle have left the farm boundaries and alerts the farmer when they do.

- **How we mapped participants:**
    1. **Singleton**: LocationDatabase class
    2. Private constructor prevents direct instantiation
    3. Static getInstance() method controls access to the single instance
    4. Thread-safe implementation with synchronized method

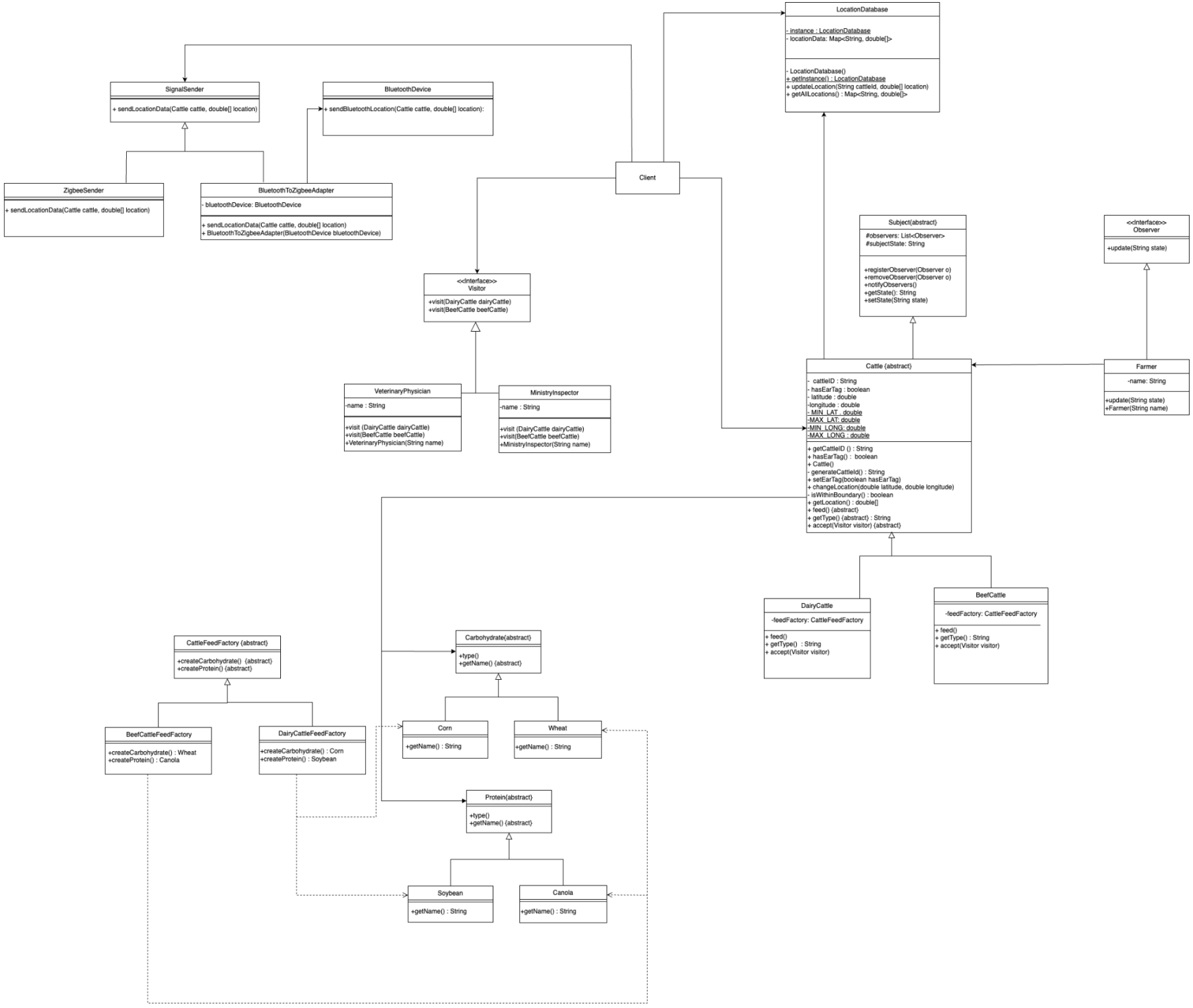
1.5. Adapter Pattern:

- **Why we chose this pattern:**

The server on the farm only accepts the Zigbee signals. Few Cattle are equipped with Bluetooth-based tracking devices. This pattern allows us to integrate these Bluetooth devices into the system by converting their communication into Zigbee compatible format. This enables us to use both Bluetooth and Zigbee devices without modifying the server code.

- **How we mapped participants:**
    1. **Target**: SignalSender interface
    2. **Adaptee**: BluetoothDevice class
    3. **Adapter**: BluetoothToZigbeeAdapter class
    4. **Client**: Livestock main class

2. UML Diagram


LocationDatabase
- instance : LocationDatabase
- locationData: Map<String, double[]>
- LocationDatabase()
+ getInstance() : LocationDatabase
+ updateLocation(String cattleId, double[] location)
+ getAllLocations() : Map<String, double[]>
SignalSender
+ sendLocationData(Cattle cattle, double[] location)
BluetoothDevice
+ sendBluetoothLocation(Cattle cattle, double[] location):
ZigbeeSender
+ sendLocationData(Cattle cattle, double[] location)
BluetoothToZigbeeAdapter
- bluetoothDevice: BluetoothDevice
+ sendLocationData(Cattle cattle, double[] location)
+ BluetoothToZigbeeAdapter(BluetoothDevice bluetoothDevice)
Client
<<Interface>>
Visitor
+visit(DairyCattle dairyCattle)
+visit(BeefCattle beefCattle)
Subject(abstract)
#observers: List<Observer>
#subjectState: String
+registerObserver(Observer o)
+removeObserver(Observer o)
+notifyObservers()
+getState(): String
+setState(String state)
<<Interface>>
Observer
+update(String state)
VeterinaryPhysician
-name : String
+visit (DairyCattle dairyCattle)
+visit(BeefCattle beefCattle)
+VeterinaryPhysician(String name)
MinistryInspector
-name : String
+visit (DairyCattle dairyCattle)
+visit(BeefCattle beefCattle)
+MinistryInspector(String name)
Cattle (abstract)
- cattleID : String
- hasEarTag : boolean
- latitude : double
-longitude : double
- MIN_LAT : double
-MAX_LAT: double
-MIN_LONG: double
-MAX_LONG : double
+ getCattleID () : String
+ hasEarTag() : boolean
+ Cattle()
- generateCattleId() : String
+ setEarTag(boolean hasEarTag)
+ changeLocation(double latitude, double longitude)
- isWithinBoundary() : boolean
+ getLocation() : double[]
+ feed() {abstract}
+ getType() {abstract} : String
+ accept(Visitor visitor) {abstract}
Farmer
-name: String
+update(String state)
+Farmer(String name)
DairyCattle
-feedFactory: CattleFeedFactory
+ feed()
+ getType() : String
+ accept(Visitor visitor)
BeefCattle
-feedFactory: CattleFeedFactory
+ feed()
+ getType() : String
+ accept(Visitor visitor)
CattleFeedFactory (abstract)
+createCarbohydrate() {abstract}
+createProtein() {abstract}
Carbohydrate(abstract)
+type()
+getName() {abstract}
BeefCattleFeedFactory
+createCarbohydrate() : Wheat
+createProtein() : Canola
DairyCattleFeedFactory
+createCarbohydrate() : Corn
+createProtein() : Soybean
Corn
+getName() : String
Wheat
+getName() : String
Protein(abstract)
+type()
+getName() {abstract}
Soybean
+getName() : String
Canola
+getName() : String

3. Explanation of Classes in Livestock Farm Project

3.1 Abstract Factory Pattern Classes

3.1.1 Feed Products

- **Protein (Abstract Class):**

  Every protein feed in our system will inherit from this class.

  Methods:

  1. **type():** This is the method that allows us to simply define this as a protein feed.
  2. **getName():** This is the method that allows us to return the name of each specific protein type

- **Soybean (Concrete Protein Class):**

  A specific type of protein feed used for dairy cattle.

  Methods:

  1. **getName():** Returns a String value "It is a Soybean".

- **Canola (Concrete Protein Class):**

  A specific type of protein feed used for beef cattle.

  Methods:

  1. **getName():** Returns a String value "It is a Canola".

- **Carbohydrate (Abstract Class):**

Every carbohydrate feed in our system will inherit from this class.

Methods:

1. **type():** This is the method that allows us to simply define this as a carbohydrate feed.
2. **getName():** This is the method that allows us to return the name of each specific carbohydrate type.

- **Corn (Concrete Carbohydrate):**

A specific type of carbohydrate feed used for dairy cattle.

Methods:

1. **getName():** Returns a String value ""It's Corn".

- **Wheat (Concrete Carbohydrate):**

A specific type of carbohydrate feed used for beef cattle.

Methods:

1. **getName():** Returns a String value ""It's Wheat".

3.1.2 Feed Factories

- **CattleFeedFactory (Abstract Factory):**

The abstract factory that defines what methods concrete factories must implement. This factory is responsible for creating sets of feed products.

Methods:

1. **createCarbohydrate():** Abstract method to create a carbohydrate feed.
2. **createProtein():** Abstract method to create a protein feed.

- **DairyCattleFeedFactory (Concrete Factory):**

  A specific factory that creates the right feeds for dairy cattle.

  Methods:

  1. **createCarbohydrate():** Creates corn for dairy cattle.
  2. **createProtein():** Creates soybean for dairy cattle.

- **BeefCattleFeedFactory (Concrete Factory):**

  A specific factory that creates the right feeds for beef cattle.

  Methods:

  1. **createCarbohydrate():** Creates wheat for beef cattle.
  2. **createProtein():** Creates canola for beef cattle.

3.2 Observer Pattern Classes

- **Observer (Interface):** Defines a contract for all observers to receive notifications from the subject.

  Methods:

  1. **update(String state):** Method called when the state of the subject changes state.

- **Subject (Abstract Class):** Maintains a list of observers and provides methods to register, remove, notify them when needed.

  Methods:

  1. **registerObserver(Observer o):** Adds a new observer.
  2. **removeObserver(Observer o):** Removes an observer.
  3. **notifyObservers():** Notifies all registered observers when state changes.
  4. **getState():** Returns the current state.
  5. **setState(String state):** Sets a new state.
- **Farmer (Concrete Observer Class):** A concrete observer that receives notifications when cattle cross boundaries.

  Methods:

  1. **update(String state):** Prints an alert when notified.

3.3 Visitor Pattern Classes

- **Visitor (Interface):** Defines operations to be performed on cattle.

  Methods:

  1. **visit(DairyCattle dairyCattle):** Abstract method that defines an operation to be performed on dairy cattle
  2. **visit(BeefCattle beefCattle):** Abstract method that defines an operation to be performed on beef cattle.

- **VeterinaryPhysician (Concrete Visitor Class):** Visits farm in autumn to vaccinate cattle.

  Methods:

  1. **visit(DairyCattle dairyCattle):** This method prints a message showing that the vet has vaccinated a specific dairy cattle by ID.
  2. **visit(BeefCattle beefCattle):**This method prints a message showing that the vet has vaccinated a specific beef cattle by ID.

- **MinistryInspector (Concrete Visitor Class):** Visits farm in spring to check ear tags.

  Methods:

1. **visit(DairyCattle dairyCattle):** If dairy cattle have proper ear tags. This method checks the hasEarTag() property and prints different messages based on whether the tag is present.
2. **visit(BeefCattle beefCattle):** If beef cattle have proper ear tags. This method checks the hasEarTag() property and prints different messages based on whether the tag is present.

3.4 Singleton Pattern Classes

LocationDatabase (Singleton Class): A single database that maintains the locations of all cattle using the Singleton pattern to ensure a single instance throughout the system.

Methods:

1. **getInstance():** Returns the globally shared single instance of the database.
2. **updateLocation(String cattleId, double[] location):** Updates a location of a specific cattle.
3. **getAllLocations():** Returns a map that contains all stored cattle locations.

3.5 Adapter Pattern Classes

- **SignalSender (Interface):** Defines a common interface for sending location data to the server.

  Methods:

  1. **sendLocationData(Cattle cattle, double[] location):** Sends location data.
- **ZigbeeSender (SignalSender Concrete Class):** Sends location data using Zigbee signals.

  Methods:

  1. **sendLocationData(Cattle cattle, double[] location):** Sends location data via Zigbee.
- **BluetoothDevice (Class):** Represents a device that sends location data using Bluetooth signals.

  Methods:

  1. **sendBluetoothLocation(Cattle cattle, double[] location):** Sends location via Bluetooth.

- **BluetoothToZigbeeAdapter (Class):** Converts a Bluetooth device so that it can be used where a Signal Sender is expected.

  Methods:

  1. **sendLocationData(Cattle cattle, double[] location):** Adapts device that uses Bluetooth signals to Zigbee format.

3.6 Central Participant Classes:

- **Cattle (Abstract Class):** Base class for all cattle types. It is also a Subject for observers, Client for the Abstract Factory pattern, Element for the Visitor pattern.
  1. **cattleId:** A unique ID for each cattle.
  2. **hasEarTag:** Shows if the cattle has a tracking tag.
  3. **latitude, longitude:** The current location of the cattle.
  4. **MIN/MAX_LAT/LONG:** Defines the farm boundary.

  Methods:

  1. **getCattleId():** Returns the unique cattle ID.
  2. **hasEarTag():** Checks if the cattle has an ear tag.
  3. **setEarTag(boolean):** Sets the ear tag status.
  4. **changeLocation(double latitude, double longitude):** Updates location and checks boundaries.It checks whether the animal has left the farm area. If it has, it sends a notification to all registered observers.
  5. **isWithinBoundary():** Checks if cattle is within farm boundaries.

6. **getLocation():** Returns current location.
7. **accept(Visitor visitor):** Accepts a visitor for Visitor pattern.
8. **getType():** Returns the type of cattle.
9. **feed():** Abstract method to define the feeding behavior of the cattle.

- **DairyCattle (Concrete Cattle Class):** Represents dairy cattle that produce milk.

  Methods:

  1. **accept(Visitor visitor):** Accepts a visitor operation.
  2. **getType():** Returns "Dairy Cattle".
  3. **feed():** Feeds dairy cattle with corn and soybean.

- **BeefCattle (Concrete Cattle Class):** Represents beef cattle that are raised for meat.

  Methods:

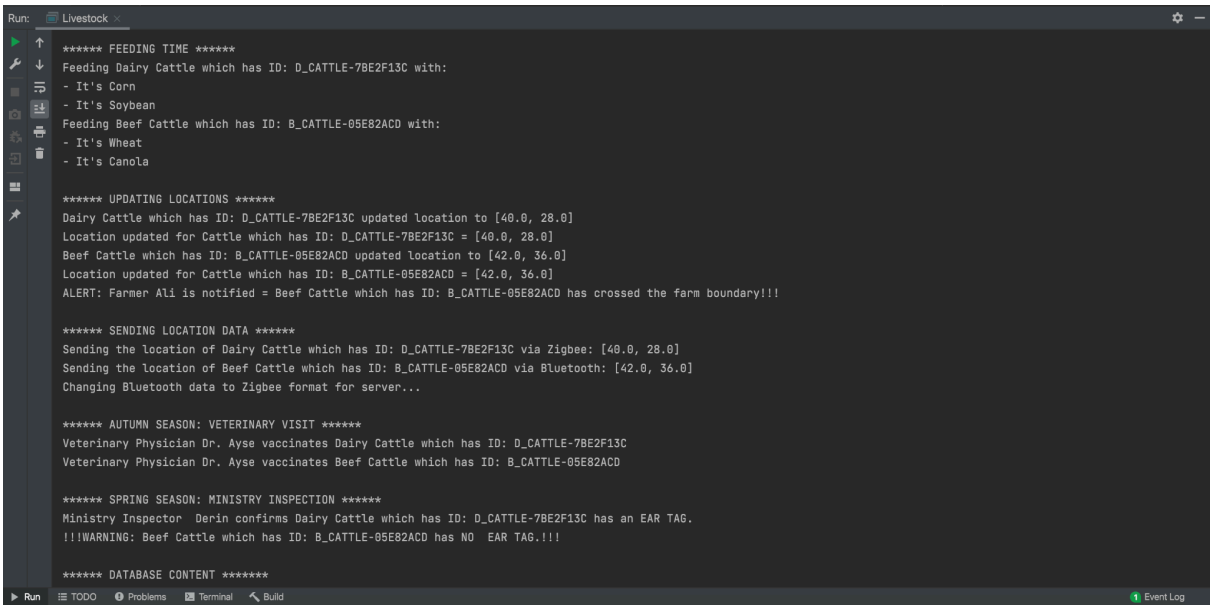
  1. **accept(Visitor visitor):** Accepts a visitor operation.
  2. **getType():** Returns "Beef Cattle".
  3. **feed():** Feeds beef cattle with wheat and canola.

3.7 Livestock(Main Class - Client)

Acts as the entry point of the application. This class creates and coordinates the main components of the system by interacting all design patterns used.

Responsibilities:

1. Creates DairyCattle and BeefCattle objects.
2. Adds Farmer instances to cattle objects to receive notifications.

3. Invokes the feed() method that uses the Abstract Factory Pattern to generate feed components.
4. Calls to changeLocation(...) are made to trigger Observer notifications when cattle cross their boundaries.
5. Uses the singleton LocationDatabase.getInstance() to retrieve all cattle locations at the end.
6. Passes VeterinaryPhysician and MinistryInspector objects to cattle via the accept(...) method, enabling external operations.(Visitor Pattern)
7. Shows how to use the Adapter Pattern by directly using Zigbee Sender and BluetoothDevice via Bluetooth to Zigbee Adapter.

4. Screenshots of Our Running Program